

there could not be a better time to learn Android.

Android architecture

Android is developed on a layered architecture, with libraries and drivers forming the lower abstraction layers, and the framework, applications, etc, forming the top layers. Each layer provides the necessary services to the layers above and below them. This layered architecture ensures that there are no ambiguities, and the app developer has access to all the APIs.



The Android app

Let's cover some basics regarding app development, so you get an idea about how it works and can plan to develop your app.

Android apps are developed in Java, so you should be clear about Java and object-oriented programming concepts (if not, at least the concepts of classes and objects should be known).

The Eclipse IDE for Android development can easily be downloaded from the Android website. This download includes the ADT (the Android Development Tools) packaged with the IDE. If you already have Eclipse, then you can download only the ADT, and install it as a plug-in for Eclipse. ADT makes sure you have all the latest packages and libraries required for Android apps, and the latest Android image for emulation.

The Eclipse IDE creates the file organisation for your app. After the app is developed, all the code, resources, and everything that is part of your app should be archived together to form a *.apk* (Android package) file. This is the file that is installed on the Android device. Eclipse does the archiving part also, so that you can focus on the app development.

During or after app development, the IDE also lets you run your app on a real Android device (if you have any) via a USB connection, or a virtual Android device. With this, you can test and debug your app very easily.

The manifest file, an XML file, is a major component of an Android app. First, it allows you to declare the permissions the app requires. The user will be prompted with all the

requested permissions during the app installation. Besides permissions, the components of the app, the minimum API level required, the hardware and software features used, and the API libraries that the application should be linked with—all these must be declared in the manifest file.

An Android app can have four types of components. It is important to understand each component type, and how it is useful, so let's briefly cover each.

- Activity:** This represents an activity done by the user in your app. For example, in a camera app, the camera viewfinder screen and the image browser screen are separate activities. An activity of your app can be invoked by other apps also, if they have the permission, and vice-versa, while the users feel as if the activity is part of the app they are using. Such architecture allows for better code and functional usage.
- Services:** These run in the background while the user continues to interact with the app in the foreground. They are used for cases where you need to fetch data, record a sound or play music in the background. As you would have guessed, a service doesn't provide a user interface; at most, there can be a notification for completion of the service.
- Content provider:** This is the component that provides data to the app, i.e., any application can query or modify a database through the content provider. So, when you have proper permissions, you can use the content provider to read or modify the user data.
- Broadcast receivers:** The system interacts with the apps via broadcasts, if required. For example, when the battery is low, or the screen gets turned off, the system issues a broadcast to all applications. The broadcast receivers respond to the broadcasts, and initiate the corresponding service.

Now that we've been through the different aspects and the basic information about Android app development, I hope you are excited about getting started on developing a real Android app, which I will begin with in the next article. Till then, get ready with ideas for what your app will do! 

By: Nitish Kumar Tiwari

The author is based in Bengaluru, India, and works as a software developer for a FOSS-based firm. In his free time, he likes to dabble with open source software and share his experiences. Other interests include movies and travelling. Contact him at Nitish.tiwari@techcube.in.

EXCELLENT EMAIL INFRASTRUCTURE SOLUTIONS WITH UNMATCHED SERVICE SUPPORT!



- TechnoMail - Enterprise Email Server Anti SPAM, Anti Virus, Email Content Filtering
- Firewall, Internet Access Control Content Filtering, Site Blocking
- Bandwidth Management System
- Managed Email Hosting Solutions

TechnoInfotech

1, Vikas Permisses, 11 Bank Street,
Fort Mumbai, India-400 001,
Mobile: 09167399917.
Email: info@technoinfotech.com